Wedge, Dual, and Binding

The cross product as a macro, and what a typed binding costs

James Pustejovsky

Brandeis University
Department of Computer Science

CS 135: Computational Semantics

Fall 2026

Where This Picks Up

Session 13a declared a quadratic form and got the Clifford product from it. Session 13b read products of vectors as programs, with Q as the interpreter: two rewrite rules evaluate every product. Today the interpreter is fixed, Euclidean three-space, and we build two *typed* functions on top of the product.

The claim for today

Geometric algebra is a small functional language for space. It has one instruction, the geometric product; two typed functions built from it, `wedge`, a **constructor**, and `dual`, a **cast**; and the familiar cross product is a **macro** written in terms of the two:

$$\texttt{cross a b} = \texttt{dual}\,(\texttt{wedge a b})$$

Reading the types of `wedge` and `dual` answers three questions about the cross product that calculus courses leave open. It also says exactly what HRR gives up when it binds two vectors and gets a vector back (Session 12), and what FGA keeps instead.

Roadmap

I	Three questions	why only 3D, why a right-hand rule, why not associative
II	The machine and its types	one instruction; the plane's i is a bivector; grades are types
III	`wedge`, a constructor	two vectors in, the program for their plane out
IV	`dual`, a cast	a k-blade renamed by its complement; a global flag
V	The macro	`cross` $=$ `dual` $\circ$ `wedge`, and the three answers
VI	Binding	HRR's cast is built in and lossy; FGA keeps the grade
VII	Close	summary; exercises; reading

Every product is evaluated by hand with the two rules of Session 13b, and every number is reproduced by a short script with the geometric product as its only primitive.

Part I
Three Questions

Three Questions a Calculus Course Never Answers

Everyone here has computed $\mathbf{a} \times \mathbf{b}$. Three things about it are usually stated and not explained:

1. Why does the cross product exist **only in three dimensions**? Dot products exist in every dimension.
2. Why does it need a **right-hand rule**? Nothing else in linear algebra asks which hand you use.
3. Why is it **not associative**? $(\mathbf{a} \times \mathbf{b}) \times \mathbf{c} \neq \mathbf{a} \times (\mathbf{b} \times \mathbf{c})$, while every other product the course relies on is associative.

By the end of Part V, all three answers fall out of the one line

$$\texttt{cross a b} = \texttt{dual}\,(\texttt{wedge a b})$$

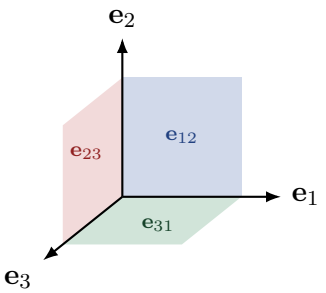
once we read the types of `wedge` and `dual`.

From the Plane to Space: One i, or Many?

Session 13b read i as the program "turn a quarter turn," and $i^2 = -1$ as running it twice. In the plane that is a complete instruction: there is only one plane to turn in.

In three dimensions it is underspecified: turn a quarter turn in *which* plane? There are three coordinate planes and infinitely many tilted ones.

So there is no single i in 3D. There is a **family of i's**, one per plane. Hamilton spent a decade looking for "the" three-dimensional number system before he saw this, in 1843.



three planes, three quarter-turn programs

What we need is a function that takes two vectors and returns the i of the plane they span. That function is the wedge.

Part II
The Machine and Its Types

One Instruction, Two Rules

The language has one primitive, the geometric product of vectors, written by juxtaposition. Today's interpreter is $\mathrm{Cl}(3,0,0)$: three orthonormal letters, each squaring to $+1$. Session 13b's two rules evaluate every product:

SWAP	$\mathbf{e}_j\mathbf{e}_i \to -\,\mathbf{e}_i\mathbf{e}_j \quad (i \neq j)$	reorder at the cost of a sign
COLLAPSE	$\mathbf{e}_i\mathbf{e}_i \to Q(\mathbf{e}_i) = +1$	a letter meeting itself returns a number

- Everything else is generated: products of letters give new elements, the **blades**, such as $\mathbf{e}_1\mathbf{e}_2 =: \mathbf{e}_{12}$ and $\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 =: \mathbf{e}_{123}$.
- Example: $\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2 \to -\,\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2 \to -\,\mathbf{e}_1$.
- The product is associative and distributes over $+$; it is not commutative, because SWAP costs a sign.

In the plane, $n = 2$

The whole algebra has four basis elements: 1, $\mathbf{e}_1$, $\mathbf{e}_2$, $\mathbf{e}_{12}$.

The Plane's i Is a Bivector

Take $\mathbf{e}_{12} = \mathbf{e}_1\mathbf{e}_2$ and run it on the basis vectors of the plane by right multiplication:

$$\mathbf{e}_1\,\mathbf{e}_{12} = \mathbf{e}_1\mathbf{e}_1\mathbf{e}_2 = \mathbf{e}_2$$
$$\mathbf{e}_2\,\mathbf{e}_{12} = \mathbf{e}_2\mathbf{e}_1\mathbf{e}_2 = -\,\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2 = -\,\mathbf{e}_1$$
$$\mathbf{e}_{12}\,\mathbf{e}_{12} = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2 = -\,\mathbf{e}_1\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2 = -1$$

- $\mathbf{e}_1 \mapsto \mathbf{e}_2$ and $\mathbf{e}_2 \mapsto -\mathbf{e}_1$: a counterclockwise quarter turn. And $\mathbf{e}_{12}^2 = -1$: run it twice, face backwards.
- So **the plane's i is $\mathbf{e}_{12}$**, the unit **bivector**. It is not a scalar and not a vector; it is the oriented unit square $\mathbf{e}_1 \wedge \mathbf{e}_2$ that the two basis vectors span.
- Left multiplication turns the other way: $\mathbf{e}_{12}\,\mathbf{e}_1 = -\,\mathbf{e}_2$.

One-sided multiplication turns only vectors that lie in the plane: $\mathbf{e}_3\,\mathbf{e}_{12} = \mathbf{e}_{123}$ is a volume, not a vector. The general way to run a bivector on any vector is Session 13b's sandwich, $R\,\mathbf{x}\,\widetilde{R}$ with $R = e^{-\mathbf{e}_{12}\theta/2}$; in the plane, for $\theta = 90°$, the two agree.

Grades Are Types

In three dimensions the algebra has $2^3 = 8$ basis blades, sorted by **grade**, the number of distinct letters that survive evaluation:

grade	type	basis	dim	picture
0	Scalar	1	1	a number
1	Vec	$\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$	3	an arrow
2	Bivec	$\mathbf{e}_{12}, \mathbf{e}_{23}, \mathbf{e}_{31}$	3	an oriented plane segment
3	Trivec	$\mathbf{e}_{123}$	1	an oriented volume

- Read a grade as a type, Blade k; Session 13b read it off the program text by counting the letters with odd multiplicity.
- The geometric product does not respect these types: for two vectors, $\mathbf{ab} = \underbrace{\mathbf{a} \cdot \mathbf{b}}_{\text{Scalar}} + \underbrace{\mathbf{a} \wedge \mathbf{b}}_{\text{Bivec}}$, a mixed-type result.
- The two functions built next are **well-typed**: each is carved out of this product by keeping one grade.

Three Planes, Three Programs

Each coordinate plane has its own unit bivector, and each one squares to -1:

$$\mathbf{e}_{12}^2 = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2 = -1 \qquad \text{“a quarter turn in the } \mathbf{e}_1\mathbf{e}_2 \text{ plane”}$$

$$\mathbf{e}_{23}^2 = \mathbf{e}_2\mathbf{e}_3\mathbf{e}_2\mathbf{e}_3 = -1 \qquad \text{“a quarter turn in the } \mathbf{e}_2\mathbf{e}_3 \text{ plane”}$$

$$\mathbf{e}_{31}^2 = \mathbf{e}_3\mathbf{e}_1\mathbf{e}_3\mathbf{e}_1 = -1 \qquad \text{“a quarter turn in the } \mathbf{e}_3\mathbf{e}_1 \text{ plane”}$$

So 3D has three independent i's, and every tilted plane has one of its own, a combination of these three. The instruction “turn a quarter turn” is incomplete until the plane is named.

Hamilton's detour (1843)

Hamilton wanted a number system for 3D as the complex numbers are for 2D. His quaternions $a + bi + cj + dk$ need four components, with $i^2 = j^2 = k^2 = ijk = -1$. Geometric algebra says why four: a quaternion is a scalar plus a bivector, and 3D has one scalar and three planes. With the signs Session 14 will use, $i = \mathbf{e}_3\mathbf{e}_2$, $j = \mathbf{e}_1\mathbf{e}_3$, $k = \mathbf{e}_2\mathbf{e}_1$, and $ij = \mathbf{e}_3\mathbf{e}_2\mathbf{e}_1\mathbf{e}_3 = \mathbf{e}_2\mathbf{e}_1 = k$: Hamilton's i, j, k are the quarter-turn programs for the three coordinate planes.

For Part V: the cross product was later extracted from the quaternion product by Gibbs and Heaviside, and inherited the algebra's oddities without the explanations.

Part III
`wedge`, a Constructor

wedge: The Type and the Definition

Type and definition

$$\texttt{wedge} \;::\; \text{Vec} \to \text{Vec} \to \text{Bivec} \qquad\qquad \mathbf{a} \wedge \mathbf{b} \;=\; \tfrac{1}{2}\,(\mathbf{ab} - \mathbf{ba}),$$

the antisymmetric half of the geometric product.

Recall the split that has run through the course since the bilinear forms of Session 10: any product of two vectors is

$$\mathbf{ab} \;=\; \underbrace{\tfrac{1}{2}(\mathbf{ab} + \mathbf{ba})}_{\text{symmetric} \,=\, \mathbf{a}\cdot\mathbf{b}} \;+\; \underbrace{\tfrac{1}{2}(\mathbf{ab} - \mathbf{ba})}_{\text{antisymmetric} \,=\, \mathbf{a}\wedge\mathbf{b}}$$

- The symmetric half is a scalar, written by the interpreter. The antisymmetric half is a bivector, written by the order of the letters (Session 13b). `wedge` keeps the second half.
- It **changes type**: two vectors in, a bivector out. That is what makes it a constructor rather than an operation on a fixed set.

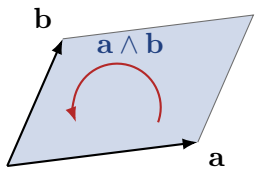
What the Constructor Returns

$$\mathbf{a} \wedge \mathbf{b} \;=\; |\mathbf{a}|\,|\mathbf{b}|\,\sin\theta\;\;\hat{B}, \qquad \hat{B}^2 = -1$$

- $\hat{B}$ is the unit bivector of the plane through $\mathbf{a}$ and $\mathbf{b}$: the plane's own i.
- $|\mathbf{a}|\,|\mathbf{b}|\sin\theta$ is the **area** of the parallelogram they span.
- The orientation, from $\mathbf{a}$ toward $\mathbf{b}$, is the **direction of turn**.

In words

`wedge a b` compiles two vectors into the quarter-turn program for their plane, with an amplitude equal to the area.



an oriented area: the magnitude is the area, the arrow the sense of turn

In the Plane, `wedge` Returns i, Scaled

In 2D there is only one bivector direction, $\mathbf{e}_{12}$, so the constructor always returns a multiple of it:

$$\mathbf{a} \wedge \mathbf{b} \;=\; (a_1 b_2 - a_2 b_1)\,\mathbf{e}_{12} \;=\; \det(\mathbf{a}, \mathbf{b})\; i$$

- The signed area is the familiar 2×2 determinant.
- Example: $\mathbf{a} = (1, 2)$, $\mathbf{b} = (3, 4)$:

  $$\mathbf{a} \wedge \mathbf{b} = (1 \cdot 4 - 2 \cdot 3)\,\mathbf{e}_{12} = -2\,\mathbf{e}_{12}.$$

  Negative: going from $\mathbf{a}$ to $\mathbf{b}$ is a clockwise turn.
- So in 2D the question "which plane?" is trivial, and `wedge` reports only **how much** and **which way**. In 3D it also reports **which plane**, and that is where the type Bivec starts doing work.

A Worked Example in 3D

Let $\mathbf{a} = (1, 2, 3)$ and $\mathbf{b} = (4, 5, 6)$. Expand by bilinearity, with $\mathbf{e}_i \wedge \mathbf{e}_i = 0$ and $\mathbf{e}_j \wedge \mathbf{e}_i = -\,\mathbf{e}_i \wedge \mathbf{e}_j$, in the basis $\mathbf{e}_{12}, \mathbf{e}_{23}, \mathbf{e}_{31}$:

$$\mathbf{a} \wedge \mathbf{b} = (a_1 b_2 - a_2 b_1)\,\mathbf{e}_{12} + (a_2 b_3 - a_3 b_2)\,\mathbf{e}_{23} + (a_3 b_1 - a_1 b_3)\,\mathbf{e}_{31}$$

plane	coefficient	
$\mathbf{e}_{12}$	$1 \cdot 5 - 2 \cdot 4$	-3
$\mathbf{e}_{23}$	$2 \cdot 6 - 3 \cdot 5$	-3
$\mathbf{e}_{31}$	$3 \cdot 4 - 1 \cdot 6$	$+6$

$$\boxed{\mathbf{a} \wedge \mathbf{b} = -3\,\mathbf{e}_{12} - 3\,\mathbf{e}_{23} + 6\,\mathbf{e}_{31}}$$

- Three numbers, one per coordinate plane: how much of the parallelogram's oriented area projects onto each plane.
- Keep these three numbers; they return when we cast this bivector to a vector.

The cyclic basis $\mathbf{e}_{12}, \mathbf{e}_{23}, \mathbf{e}_{31}$ is the one Session 14 uses throughout. In the basis $\mathbf{e}_{13} = -\mathbf{e}_{31}$ the last coefficient reads -6.

Properties of the Constructor, 1: The Orientation Bit

$$\mathbf{a} \wedge \mathbf{b} = -\left(\mathbf{b} \wedge \mathbf{a}\right) \qquad \text{anticommutative}$$

$$\mathbf{a} \wedge \mathbf{a} = 0 \qquad \text{a vector spans no area with itself}$$

$$\left(\lambda\mathbf{a} + \mu\mathbf{a}'\right) \wedge \mathbf{b} = \lambda\left(\mathbf{a} \wedge \mathbf{b}\right) + \mu\left(\mathbf{a}' \wedge \mathbf{b}\right) \qquad \text{bilinear}$$

- The sign is not a nuisance; it is **one bit of information**: which argument came first.
- Compare the dot product, which is symmetric and so cannot tell $\mathbf{a} \cdot \mathbf{b}$ from $\mathbf{b} \cdot \mathbf{a}$.
- For this course it matters: a binding operation that is anticommutative records **argument order** in the sign. Part VI compares this with HRR, whose binding is commutative.

Properties of the Constructor, 2: It Forgets on Purpose

$$\mathbf{a} \wedge (\mathbf{b} + \lambda \mathbf{a}) \;=\; \mathbf{a} \wedge \mathbf{b} + \lambda\,(\mathbf{a} \wedge \mathbf{a}) \;=\; \mathbf{a} \wedge \mathbf{b}$$

- Sliding $\mathbf{b}$ along $\mathbf{a}$ changes neither the parallelogram's area, nor its plane, nor its orientation, so it does not change $\mathbf{a} \wedge \mathbf{b}$.
- The constructor **normalizes its inputs**: many pairs $(\mathbf{a}, \mathbf{b})$ map to the same bivector. Only the plane, the orientation, and the area survive.
- Compare a constructor in a functional language that builds a set from a list: order and duplicates are discarded because the type says they do not matter.
- Consequence: $\mathbf{a}$ and $\mathbf{b}$ cannot be recovered from $\mathbf{a} \wedge \mathbf{b}$ alone. But $\mathbf{b}$ *can* be recovered if $\mathbf{a}$ is still at hand and $\mathbf{b} \perp \mathbf{a}$. Part VI uses exactly that for unbinding.

Properties of the Constructor, 3: It Is Associative

The wedge extends to blades of any grade, adding grades, and it is associative:

$\texttt{wedge} :: \text{Blade } r \to \text{Blade } s \to \text{Blade } (r+s), \quad (\mathbf{a} \wedge \mathbf{b}) \wedge \mathbf{c} = \mathbf{a} \wedge (\mathbf{b} \wedge \mathbf{c}) = \mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c} \in \text{Trivec}$

- $\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c}$ is the **oriented volume** of the parallelepiped the three vectors span.
- With $\mathbf{a} = (1,2,3)$, $\mathbf{b} = (4,5,6)$, $\mathbf{c} = (7,8,10)$:

$$\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c} = \det \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{pmatrix} \mathbf{e}_{123} = -3\,\mathbf{e}_{123},$$

  and it does not matter which pair is wedged first.
- Hold this: the cross product will turn out to **lose** associativity, and the reason is not in the wedge.

Grounding: Role–Filler Binding as a Constructor

Session 12 bound a role to a filler with circular convolution, and the wedge offers another way:

$$\mathbf{agt} \circledast \mathbf{mary} \in \text{Vec} \qquad \mathbf{agt} \wedge \mathbf{mary} \in \text{Bivec}$$

- The HRR binding has the **same type** as its arguments. The wedge binding has a **new type**: the result is a bound pair, and the type says so. Nothing in a Vec records that it was built by binding.
- The orientation bit: $\mathbf{dog} \wedge \mathbf{man} = -(\mathbf{man} \wedge \mathbf{dog})$. Circular convolution is commutative, $\mathbf{dog} \circledast \mathbf{man} = \mathbf{man} \circledast \mathbf{dog}$, so an unlabeled HRR pair cannot say who bit whom; HRR adds role vectors to say it.
- Role vectors are still useful with the wedge, for naming roles and for more than two arguments. The point is **where the information lives**: in the type and the sign, or in an extra operand.

wedge in Code

A multivector is a dictionary from basis blades (bitmasks over $\mathbf{e}_1 \dots \mathbf{e}_n$) to coefficients. The geometric product gp is the only primitive; everything else is written on top of it.

```
def gp(x, y):           # the one instruction: SWAP and COLLAPSE, about ten lines
    ...

def project(x, k):      # grade projection  <x>_k
    return {b: c for b, c in x.items() if bin(b).count("1") == k}

def wedge(x, y):        # Blade r -> Blade s -> Blade (r+s)
    """Keep only the top-grade part of the geometric product.
    For two vectors this is (xy - yx)/2."""
    return project(gp(x, y), grade(x) + grade(y))
```

```
>>> a, b = vec(1,2,3), vec(4,5,6)
>>> show(wedge(a, b))
'-3 e12 + -3 e23 + 6 e31'
```

Part IV
`dual`, a Cast

The i of the Whole Space: The Pseudoscalar

Multiply all three letters together:

$$I \;=\; \mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 \;=\; \mathbf{e}_{123} \qquad \text{(the unit oriented volume)}$$

Two facts, both by the two rules:

$$I^2 = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_3\,\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 = \mathbf{e}_1\mathbf{e}_1\,\mathbf{e}_2\mathbf{e}_3\mathbf{e}_2\mathbf{e}_3 = -1$$

$$\mathbf{e}_1 I = \mathbf{e}_1\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 = \mathbf{e}_{23}, \qquad I\mathbf{e}_1 = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_3\mathbf{e}_1 = \mathbf{e}_1\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 = \mathbf{e}_{23}$$

- $I^2 = -1$: the volume element is another square root of -1. It is the i of the **whole space**, not of any one plane.
- I commutes with every vector in 3D: moving a letter past the other two costs two signs, which cancel. That is special to odd dimensions, and it is what makes the cast below clean.
- Since $I^2 = -1$, the inverse is $I^{-1} = -I$.

In the first line, moving the second $\mathbf{e}_1$ left past $\mathbf{e}_3$ and $\mathbf{e}_2$ costs two signs; then $\mathbf{e}_2\mathbf{e}_3\mathbf{e}_2\mathbf{e}_3 = -1$ by one swap and two collapses.

What I Does to a Vector

Multiply each basis vector by I:

$$\mathbf{e}_1 I = \mathbf{e}_1\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 = \mathbf{e}_{23}$$
$$\mathbf{e}_2 I = \mathbf{e}_2\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 = -\,\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2\mathbf{e}_3 = -\,\mathbf{e}_1\mathbf{e}_3 = \mathbf{e}_{31}$$
$$\mathbf{e}_3 I = \mathbf{e}_3\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_3\mathbf{e}_3 = \mathbf{e}_{12}$$

axis $\longleftrightarrow$	**plane perpendicular to it**
$\mathbf{e}_1$	$\mathbf{e}_{23}$
$\mathbf{e}_2$	$\mathbf{e}_{31}$
$\mathbf{e}_3$	$\mathbf{e}_{12}$

- Multiplying by I turns an **axis** into **the quarter-turn program about that axis**, and multiplying by I^{-1} turns the program back into its axis.
- This is a pairing between Vec and Bivec. In 3D both have dimension 3, so the pairing is a bijection. The whole story of the cross product hinges on that coincidence.

dual: The Type and the Definition

Type and definition

$$\texttt{dual}_n \;::\; \text{Blade } k \to \text{Blade } (n-k) \qquad\qquad \texttt{dual}_n(x) \;=\; x\,I^{-1}, \quad I = \mathbf{e}_1\mathbf{e}_2\cdots\mathbf{e}_n$$

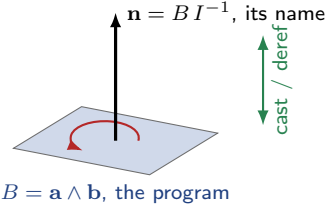
Check it on the bivectors, in 3D, where $I^{-1} = -I$:

$$\mathbf{e}_{12}\,I^{-1} = -\,\mathbf{e}_1\mathbf{e}_2\,\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 = -\,(-\,\mathbf{e}_1\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2\mathbf{e}_3) = \mathbf{e}_3, \qquad \mathbf{e}_{23}\,I^{-1} = \mathbf{e}_1, \qquad \mathbf{e}_{31}\,I^{-1} = \mathbf{e}_2$$

- A k-blade goes to the $(n-k)$-blade **orthogonal** to it, carrying the same magnitude and an orientation fixed by I.
- In 3D: planes $\to$ axes, axes $\to$ planes, scalars $\to$ volumes, volumes $\to$ scalars.
- Session 14 will meet the same operation as contraction into the pseudoscalar, $\mathbf{a} \rfloor I$, which is the general form.

No Geometry Happens in a Cast

- `dual` does not rotate, scale, or project anything. It **re-encodes**: the same oriented magnitude, stored in the complementary subspace.
- In programming terms it is a `newtype` or a `reinterpret_cast`: a change of **representation**, not of **content**.
- It is invertible, so it is **lossless**: in 3D, $x\,I^{-1}I^{-1} = -x$, and multiplying by I undoes it.
- The program $B = \mathbf{a} \wedge \mathbf{b}$ and its name $\mathbf{n} = B\,I^{-1}$ carry the same information. **Dereferencing** the name returns the program: $\mathbf{n}\,I = B$.



same information, two encodings

The Cast Depends on a Global Constant: Handedness

- The definition $\texttt{dual}_n(x) = x\,I^{-1}$ uses $I = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_3$. Nothing forces that choice: $-I = \mathbf{e}_2\mathbf{e}_1\mathbf{e}_3$ is an equally good unit volume, the **mirror-image** convention.
- Swap I for $-I$ and every dual flips sign:

$$\mathbf{e}_{12}\,(-I)^{-1} = -\,\mathbf{e}_3.$$

- So the cast has a **hidden global parameter**, like a compiler flag for byte order. Two systems using opposite flags agree on every bivector and disagree on every "axis."
- That flag has a familiar name: **the right-hand rule**. The right-hand rule is nothing but the choice of sign for the pseudoscalar.
- Handedness is not in the geometry of $\mathbf{a}$ and $\mathbf{b}$. It enters only when their plane is **cast** to a name.

The Type Says What `dual` Returns in Each Dimension

$$\texttt{dual}_n \;::\; \text{Blade } 2 \to \text{Blade } (n-2)$$

n	$\texttt{dual}_n$ **of a bivector**	**example**	**what you get**
2	Bivec $\to$ Scalar	$\texttt{dual}_2(-2\,\mathbf{e}_{12}) = -2$	a signed number
3	Bivec $\to$ Vec	$\texttt{dual}_3(\mathbf{e}_{12}) = \mathbf{e}_3$	an axis
4	Bivec $\to$ Bivec	$\texttt{dual}_4(\mathbf{e}_{12}) = -\,\mathbf{e}_{34}$	another plane
n	Bivec $\to$ Blade $(n-2)$		an $(n-2)$-blade

- The same one-line program, run in different dimensions, returns different types.
- Only at $n = 3$ does the dual of a plane land back in Vec. This is the answer to question 1.
- In 4D the complement of a plane is a plane, so a rotation has no axis at all. Rotations in 4D have planes, not axes; so do rotations of a 768-dimensional embedding.

In 4D, $I = \mathbf{e}_{1234}$ squares to $+1$, so $I^{-1} = I$ and $\mathbf{e}_{12}\,\mathbf{e}_{1234} = -\,\mathbf{e}_{34}$.

Grounding: Casts and Coercions in Semantics

Natural language has casts too. In Generative Lexicon theory, **type coercion** re-encodes an argument so that a predicate can take it: in *Mary began the book*, begin wants an event and the book is an object, so the object is read as an event associated with it, reading or writing, through its qualia.

	`dual`	**GL coercion**
direction fixed by	a global convention, I	the predicate's selected type
lossless?	yes: invertible	no: picks one of several events
ambiguity	none, given I	real: read vs. write
content changed?	no	no: the book is still the book

A dual is what coercion would be if the type system were a bijection. The comparison shows what coercion **adds**: a choice, and so an ambiguity.

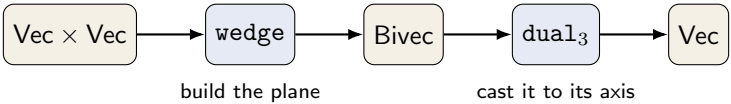
FGA models the coercion itself as a rotor, a program run on the argument (Session 13b), not as a cast. The comparison here is with the cast's losslessness only.

Part V
The Macro: The Cross Product

One Line

The cross product as a macro

$$\texttt{cross} :: \text{Vec} \to \text{Vec} \to \text{Vec} \qquad \texttt{cross a b} = \texttt{dual}_3\,(\texttt{wedge a b})$$



- Step 1 does the geometry: it finds the plane of $\mathbf{a}$ and $\mathbf{b}$, its orientation, and the area.
- Step 2 does no geometry: it renames the plane by the axis perpendicular to it.
- The cross product is not an instruction of the language. It is a **legacy macro**: a constructor followed by a cast, with the cast's global flag baked in as the right-hand rule.

Does the Macro Agree With the Textbook?

Run it on the basis vectors:

$$\mathbf{e}_1 \times \mathbf{e}_2 = \texttt{dual}_3(\mathbf{e}_1 \wedge \mathbf{e}_2) = \mathbf{e}_{12}\, I^{-1} = \mathbf{e}_3$$
$$\mathbf{e}_2 \times \mathbf{e}_3 = \texttt{dual}_3(\mathbf{e}_{23}) = \mathbf{e}_{23}\, I^{-1} = \mathbf{e}_1$$
$$\mathbf{e}_3 \times \mathbf{e}_1 = \texttt{dual}_3(\mathbf{e}_{31}) = \mathbf{e}_{31}\, I^{-1} = \mathbf{e}_2$$

One of them in full, using $I^{-1} = -I$ and the two rules:

$$\begin{aligned} \mathbf{e}_{23}\, I^{-1} &= -\,\mathbf{e}_2\mathbf{e}_3\, \mathbf{e}_1\mathbf{e}_2\mathbf{e}_3 = -\,\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3\, \mathbf{e}_2\mathbf{e}_3 \qquad \text{(two swaps to move } \mathbf{e}_1 \text{ left: no net sign)} \\ &= -\,\mathbf{e}_1\,(\mathbf{e}_2\mathbf{e}_3\mathbf{e}_2\mathbf{e}_3) = -\,\mathbf{e}_1\,(-1) = \mathbf{e}_1. \end{aligned}$$

- These are the textbook's $\hat{\imath} \times \hat{\jmath} = \hat{k}$, $\hat{\jmath} \times \hat{k} = \hat{\imath}$, $\hat{k} \times \hat{\imath} = \hat{\jmath}$.
- The cyclic pattern students memorize is the pairing table "axis $\leftrightarrow$ perpendicular plane" from Part IV.

The Numeric Check

Take $\mathbf{a} = (1, 2, 3)$, $\mathbf{b} = (4, 5, 6)$ again. We already have the plane:

$$\mathbf{a} \wedge \mathbf{b} = -3\,\mathbf{e}_{12} - 3\,\mathbf{e}_{23} + 6\,\mathbf{e}_{31}.$$

Cast each term with the table $\mathbf{e}_{12} \mapsto \mathbf{e}_3$, $\mathbf{e}_{23} \mapsto \mathbf{e}_1$, $\mathbf{e}_{31} \mapsto \mathbf{e}_2$:

$$\begin{aligned} -3\,\mathbf{e}_{12} &\mapsto -3\,\mathbf{e}_3 \\ -3\,\mathbf{e}_{23} &\mapsto -3\,\mathbf{e}_1 \\ +6\,\mathbf{e}_{31} &\mapsto +6\,\mathbf{e}_2 \end{aligned}$$

$$\boxed{\mathbf{a} \times \mathbf{b} = -3\,\mathbf{e}_1 + 6\,\mathbf{e}_2 - 3\,\mathbf{e}_3 = (-3,\ 6,\ -3)}$$

- `numpy.cross([1,2,3],[4,5,6])` returns `[-3, 6, -3]`.
- The three numbers of the cross product **are** the three numbers of the bivector, relabeled. The cross product has three components because 3D has three planes, not because the result is "really" an arrow.

Answer 1: Why Only in Three Dimensions

The macro returns a Vec only when $\texttt{dual}_n$ sends Bivec to Vec, that is, when $n - 2 = 1$. Equivalently, when the space of planes has the same dimension as the space of lines:

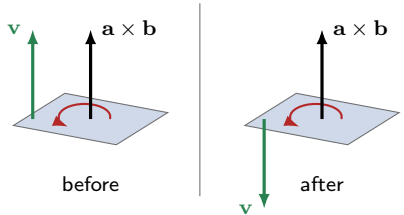
$$\dim \text{Bivec} = \binom{n}{2} \stackrel{?}{=} n = \dim \text{Vec}$$

n	$\binom{n}{2}$	n	
2	1	2	too few planes: the "cross product" is a scalar
3	3	3	match: planes and axes are in bijection
4	6	4	too many planes: no axis for a rotation
7	21	7	the 7D "cross product" is a different macro, from the octonions

The wedge exists in every dimension, and so does the dual. Their composition type-checks as $\text{Vec} \to \text{Vec} \to \text{Vec}$ only at $n = 3$. The cross product is a coincidence of dimension counting, and the oddities we are about to explain are the price of relying on a coincidence.

Answer 2: Why the Right-Hand Rule

- Reflect space in the plane of $\mathbf{a}$ and $\mathbf{b}$. The vectors $\mathbf{a}$, $\mathbf{b}$ are unchanged, so their plane $\mathbf{a} \wedge \mathbf{b}$ is unchanged.
- A true arrow $\mathbf{v}$ perpendicular to the mirror **flips**: that is what reflection does to arrows.
- But $\mathbf{a} \times \mathbf{b}$, computed from the unchanged $\mathbf{a}$, $\mathbf{b}$ with the same flag I, does **not** flip. It does not transform like an arrow: a **pseudovector**.



reflection in the plane of $\mathbf{a}, \mathbf{b}$: the arrow $\mathbf{v}$ flips, the computed $\mathbf{a} \times \mathbf{b}$ does not

In the macro this is no mystery: a reflection sends $I \mapsto -I$. The sign physics attributes to "axial vectors" lives in the **global flag of the cast**, not in the geometry. Keep the bivector and there is no rule to remember: planes reflect like planes.

Answer 3: Why It Is Not Associative

The wedge is associative. The macro is not, because it **casts in the middle of the pipeline**:

$$(\mathbf{a} \times \mathbf{b}) \times \mathbf{c} \;=\; \texttt{dual}\big(\texttt{wedge}\big(\underbrace{\texttt{dual}(\texttt{wedge}(\mathbf{a}, \mathbf{b}))}_{\text{the } \textit{name} \text{ of the plane}},\; \mathbf{c}\big)\big)$$

The second `wedge` is handed the axis of the first plane, not the plane, and spans a new plane with $\mathbf{c}$ that has nothing to do with $\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c}$.

typed pipeline: cast once, at the end	**untyped macro: cast at every step**
$(\mathbf{a} \wedge \mathbf{b}) \wedge \mathbf{c} = -3\,\mathbf{e}_{123}$	$(\mathbf{a} \times \mathbf{b}) \times \mathbf{c} = (84,\ 9,\ -66)$
$\mathbf{a} \wedge (\mathbf{b} \wedge \mathbf{c}) = -3\,\mathbf{e}_{123}$	$\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (-12,\ 9,\ -2)$

($\mathbf{a} = (1, 2, 3)$, $\mathbf{b} = (4, 5, 6)$, $\mathbf{c} = (7, 8, 10)$ as before.) The non-associativity is not a property of space. It is an artifact of throwing away the type between two steps.

The Scalar Triple Product Behaves Because It Casts Last

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) \;=\; \mathtt{dual}_3\,(\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c}) \;=\; \det[\,\mathbf{a}\ \mathbf{b}\ \mathbf{c}\,]$$

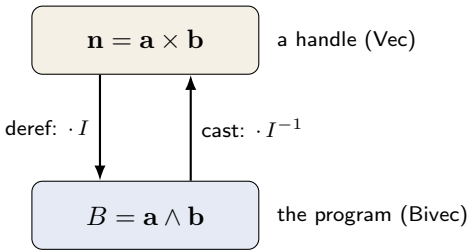
- With our vectors: $\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c} = -3\,\mathbf{e}_{123}$, and $\mathtt{dual}_3(-3\,\mathbf{e}_{123}) = -3\,\mathbf{e}_{123}\,I^{-1} = -3$. A direct computation of $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$ gives -3 as well.
- The triple product is well behaved (cyclic, equal to a determinant, independent of bracketing) because the typed representation is kept **all the way through** and cast to a number only at the end.

The functional-programming lesson

Keep the typed representation through the pipeline; **cast at the boundary**. Every oddity of the cross product comes from casting early.

The Handle and the Program

- $\mathbf{n} = \mathbf{a} \times \mathbf{b}$ is a **handle**: a name for the rotation program $B = \mathbf{a} \wedge \mathbf{b}$. Dereferencing recovers the program: $\mathbf{n}\, I = B$.
- Physics gets away with an angular velocity *vector* $\boldsymbol{\omega}$ because 3D is the one dimension where every plane has an axis. The rotation itself is executed by the bivector $\Omega = I\boldsymbol{\omega}$, through the exponential $e^{-\Omega t/2}$ of Session 13b, not by the arrow.
- Nothing can be rotated *with* $\boldsymbol{\omega}$ until it is dereferenced. Like a file handle, the arrow is small and has the same type as every other handle, which is convenient, but it carries no behavior of its own.



The "2D Cross Product" Is the Same Macro

Game programmers and computational geometers use a *2D cross product* that returns a number:

```
def cross2d(a, b):              # "is c to the left or the right of the line ab?"
    return a.x * b.y - a.y * b.x
```

- This is $\texttt{dual}_2(\mathbf{a} \wedge \mathbf{b})$: the same macro, run with $n = 2$, where the type says Bivec $\to$ Scalar.
- $\mathbf{a} = (1, 2)$, $\mathbf{b} = (3, 4)$: $\mathbf{a} \wedge \mathbf{b} = -2\,\mathbf{e}_{12}$, and $\texttt{dual}_2(-2\,\mathbf{e}_{12}) = -2\,\mathbf{e}_{12}\,\mathbf{e}_{12}^{-1} = -2$.
- Likewise the determinant in any dimension is $\texttt{dual}_n$ of the top-grade wedge $\mathbf{a}_1 \wedge \cdots \wedge \mathbf{a}_n$.
- So three things taught as separate facts, the 2D cross product, the 3D cross product, and the determinant, are **one program with three type instantiations**.

A Short History of the Legacy Macro

year	who	what
1831	Gauss	i as a rotation of the plane
1843	Hamilton	quaternions: a scalar plus the three planes of 3D
1844	Grassmann	the exterior (wedge) product, in any dimension
1878	Clifford	the geometric product, uniting Hamilton and Grassmann
1880s–1901	Gibbs, Heaviside	$\mathbf{a} \cdot \mathbf{b}$ and $\mathbf{a} \times \mathbf{b}$ extracted from the quaternion product

- Gibbs and Heaviside wanted something engineers could compute with in 3D, and they got it: vector analysis won, and the cross product became the interface everyone learned.
- The cross product is not wrong. It is an **optimization for one dimension**, compiled from a richer source and shipped without it. What was lost was the type information, and with it the explanations.
- Clifford's algebra is the source. When more than 3D, or more than one grade, is needed, we go back to the source.

The Three Programs, Side by Side

program	type	kind	what it does
`gp`	Blade $r \to$ Blade $s \to$ mixed	the instruction	the one primitive
`wedge`	Blade $r \to$ Blade $s \to$ Blade $(r+s)$	constructor	builds the plane (or volume)
$\texttt{dual}_n$	Blade $k \to$ Blade $(n-k)$	cast	renames by the complement
`cross`	Vec $\to$ Vec $\to$ Vec, only $n=3$	macro	$\texttt{dual}_3 \circ \texttt{wedge}$

question	answer
why only 3D?	$\binom{n}{2} = n$ only at $n=3$: the cast lands in Vec only there
why a right-hand rule?	it is the sign of I, the cast's global flag
why not associative?	the macro casts in the middle; the wedge never does

The Macro in Code, and Its Tests

```
def dual(x, n, handedness=+1):   # Blade k -> Blade (n-k):  x * I^{-1}
    I = pseudoscalar(n, handedness)
    return gp(x, scale(gp(I, I)[0], I))      # I^{-1} = I / I^2, and I^2 is +1 or -1

def cross(a, b, handedness=+1):  # Vec -> Vec -> Vec
    B = wedge(a, b)              # Blade 2
    return dual(B, 3, handedness)   # Blade (3 - 2) = Blade 1   <-- only in 3D
```

```
e1 x e2 = e3      e2 x e3 = e1      e3 x e1 = e2
a ^ b      = -3 e12 + -3 e23 + 6 e31
a x b      = -3 e1 + 6 e2 + -3 e3        (numpy: [-3, 6, -3])
(a x b) I == a ^ b ?  True               (dereferencing the handle gives the plane)
left-handed e1 x e2 = -e3
(a^b)^c    = -3 e123   a^(b^c) = -3 e123   dual(a^b^c) = -3 = det[a b c]
(axb)xc    = 84 e1 + 9 e2 + -66 e3        ax(bxc) = -12 e1 + 9 e2 + -2 e3
2D: dual(a^b) = -2        4D: dual(e1^e2) = -e34
```

Everything in Parts III–V is reproduced by `ga_programs.py`, with `gp` as the only primitive.

Part VI
Binding

HRR's cast is built in and lossy; FGA keeps the grade

Binding in HRR, Recalled

Session 12 bound a role to a filler with circular convolution:

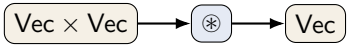
$$\mathbf{r} \circledast \mathbf{f}, \qquad (\mathbf{r} \circledast \mathbf{f})_k = \sum_j r_j \, f_{(k-j) \bmod d}$$

- Type: Vec $\to$ Vec $\to$ Vec. Roles, fillers, bound pairs, and whole sentences all live in $\mathbb{R}^d$.
- **Superpose** with $+$: $\mathbf{S} = \mathbf{agt} \circledast \mathbf{mary} + \mathbf{rec} \circledast \mathbf{john} + \mathbf{th} \circledast \mathbf{book}$ (*Mary gave John a book*).
- **Unbind** with correlation, $\mathbf{r} \# \mathbf{S} = \mathbf{r}^* \circledast \mathbf{S}$, where the involution reverses the coordinates, then clean up by nearest neighbor in the lexicon:

$$\mathbf{agt} \# \mathbf{S} \;\approx\; \mathbf{mary} + \text{noise}$$

- Circular convolution is commutative and associative, and the result has the width of its inputs. That is what makes superposition and cleanup work in one space.

The Typing of HRR Binding



everything stays in $\mathbb{R}^d$

What the design buys

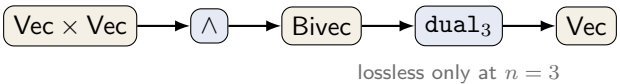
- one space for words, pairs, sentences
- superposition is just $+$
- one cleanup memory serves everything
- fixed memory cost, however much is bound

What it costs

- a Vec does not say how it was made: filler, bound pair, and superposition have one type
- commutativity: order must be added back with role vectors
- unbinding is approximate, hence the noise and the cleanup

None of these costs is a flaw; they are the terms of the trade. The question is what the alternative trade looks like.

The Cross Product as a Limiting Case of Binding



- The cross product is a **typed binding** (Vec → Vec → Bivec) followed by a **lossless cast back to** Vec. It is the one case where "bind two vectors, get a vector of the same size" loses nothing.
- It loses nothing only because $\binom{3}{2} = 3$. In $d = 10{,}000$ dimensions, Bivec has about 5×10^7 dimensions and Vec has 10^4. There is no lossless cast.
- So any Vec → Vec → Vec binding in high dimension must either be **lossy** (HRR: it is, and the noise is the loss) or **keep the grade** and give up the single space.
- That is the fork in the road. HRR takes one branch; FGA takes the other.

The FGA Move: Keep the Grade

- Bind with the wedge: $\mathbf{r} \wedge \mathbf{f} \in$ Bivec. The **type** records that this is a bound pair; the **sign** records the order; the **grade** records how many things were bound ($\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c}$ is grade 3).
- Superpose bound pairs with $+$ as before: a sentence is a bivector,

$$\mathbf{S} = \mathbf{agt} \wedge \mathbf{mary} + \mathbf{rec} \wedge \mathbf{john} + \mathbf{th} \wedge \mathbf{book}.$$

- **Unbind by contraction** with the role. For vectors, $\mathbf{r} \rfloor (\mathbf{r} \wedge \mathbf{f}) = (\mathbf{r} \cdot \mathbf{r})\,\mathbf{f} - (\mathbf{r} \cdot \mathbf{f})\,\mathbf{r}$. If the roles are unit vectors, mutually orthogonal, and orthogonal to the fillers, then

$$\mathbf{agt} \rfloor \mathbf{S} \;=\; \mathbf{mary} \qquad \text{exactly, with no noise and no cleanup.}$$

- `dual` is still available, as an **explicit cast** when a bound pair has to be compared with a vector (for scoring, say). It is not hidden inside the binding.

Worked: Binding With the Wedge, Unbinding Exactly

Roles in a subspace of their own, orthogonal to the entities: $\mathbf{r}_{\text{ag}} = \mathbf{e}_1$, $\mathbf{r}_{\text{th}} = \mathbf{e}_2$; entities john $= \mathbf{e}_3$ and mary $= 0.6\,\mathbf{e}_3 + 0.8\,\mathbf{e}_4$, with $\cos(\text{john}, \text{mary}) = 0.6$.

$$\mathbf{E} = \mathbf{r}_{\text{ag}} \wedge \text{john} + \mathbf{r}_{\text{th}} \wedge \text{mary} = \mathbf{e}_1\mathbf{e}_3 + 0.6\,\mathbf{e}_2\mathbf{e}_3 + 0.8\,\mathbf{e}_2\mathbf{e}_4$$

Since roles are orthogonal to fillers, $\mathbf{r}\,\mathbf{f} = \mathbf{r} \wedge \mathbf{f}$: a role times a filler is pure wedge. To unbind, contract with the role (its own inverse, since $Q(\mathbf{r}) = 1$):

$$\mathbf{r} \rfloor (\mathbf{r} \wedge \mathbf{f}) = \mathbf{f} - (\mathbf{r} \cdot \mathbf{f})\,\mathbf{r} = \mathbf{f} \qquad \text{exactly when } \mathbf{r} \perp \mathbf{f}$$

query	*John loves Mary*, $\mathbf{E}$	*Mary loves John*, $\mathbf{E}'$
$\mathbf{r}_{\text{ag}} \rfloor (\cdot)$	$\mathbf{e}_3 =$ john, exactly	$0.6\,\mathbf{e}_3 + 0.8\,\mathbf{e}_4 =$ mary, exactly
$\mathbf{r}_{\text{th}} \rfloor (\cdot)$	mary, exactly	john, exactly

$\cos(\mathbf{E}, \mathbf{E}') = 0.6$: the similarity of the two fillers and nothing more. No cleanup step. The MAP record of Session 12 returned its fillers at cosine 0.50 and needed one. The FGA position paper states this as a proposition: orthonormal roles orthogonal to the entity space suffice, whatever the fillers.

The FGA Move in Code

Roles in one subspace $(\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3)$, fillers in another $(\mathbf{e}_4, \mathbf{e}_5, \mathbf{e}_6)$. The left contraction of a vector into a bivector is the grade-1 part of their geometric product.

```
def lcontract(a, B):                # Vec -> Bivec -> Vec
    return project(gp(a, B), 1)

agent, recip, theme = vec(1,0,0,0,0,0), vec(0,1,0,0,0,0), vec(0,0,1,0,0,0)
mary,  john,  book  = vec(0,0,0,1,0,0), vec(0,0,0,0,1,0), vec(0,0,0,0,0,1)
S = add(add(wedge(agent, mary), wedge(recip, john)), wedge(theme, book))
```

```
>>> lcontract(agent, S) == mary,  lcontract(theme, S) == book
(True, True)
>>> wedge(dog, man) == scale(-1, wedge(man, dog))     # the orientation bit
True
```

The exactness depends on the orthogonality assumptions. With random high-dimensional vectors, orthogonality is approximate and some noise returns; how much, compared with HRR's noise at the same dimension, is an empirical question (project direction 1 below).

Why a Relation Needs Two Slots

A role–filler pair is one wedge. A *relation* between two entities is more demanding. Take the attention head of Session 11, $G = \left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right) = S + A$ with $S = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$, $A = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$, on tokens $\mathbf{x}_1 = (3, 1)$, $\mathbf{x}_2 = (1, 3)$. Three ways to store it:

store	**holds**	**numbers**	$(\mathbf{x}_1, \mathbf{x}_2)$	$(\mathbf{x}_2, \mathbf{x}_1)$	$(\mathbf{x}_1, \mathbf{x}_1)$
a symmetric form, all Q can see	S	$\frac{1}{2}n(n+1)$	16	16	16
a bivector in one entity space	A	$\binom{n}{2}$	8	-8	0
a cross-slot bivector, $V_E^s \wedge V_E^o$	G	n^2	24	8	16

One entity space is not enough. A bivector there holds only the antisymmetric half: the swap merely flips the sign, and a reflexive scores $\mathbf{x}^\top A \mathbf{x} = 0$, so *John loves himself* would vanish. FGA gives each argument an isometric copy of the entity space, a subject slot V_E^s and an object slot V_E^o, entered through injections ι_s and ι_o:

$$L = \textstyle\sum_{i,k} G_{ik}\, \mathbf{e}_i^s \wedge \mathbf{e}_k^o, \qquad \iota_s(\mathbf{x}) \rfloor \big(L \lfloor \iota_o(\mathbf{y})\big) = \mathbf{x}^\top G\, \mathbf{y}$$

Q sees only S; one space holds only A; two slots hold both. The price is n^2 numbers, as for a tensor: the $\binom{n}{2}$ saving came from keeping only the alternating sector.

Grades and Subspaces as Types

Session 11b mapped semantic types to tensors of rising order. FGA maps them to **grades** of one algebra, and adds **subspace support**, because grade alone cannot separate e from $\langle e, t\rangle$:

type	example	tensor framework	FGA: grade, and where it lives
t	a truth value	order 0	grade 0, a scalar
e	Mary	order 1	grade 1, in V_E
$\langle e, t\rangle$, verb	laughs	order 1	grade 1, in $V_E \oplus V_P$
$\langle e, t\rangle$, noun	dog	order 1	grade 1, in $V_E \oplus V_K \oplus V_P$
$\langle e, \langle e, t\rangle\rangle$	loves	order 2, n^2 numbers	grade 2, in $V_E^s \wedge V_E^o$, n^2 numbers
k-place	gives, $k = 3$	order k, n^k numbers	grade k, one slot per argument

Application is still contraction. A predicate's V_E part meets the entity, as $\mathbf{p}^\top \mathbf{x}$ did in Session 6; its V_P content drops out, since the subspaces are mutually orthogonal. A noun's V_K part, which a verb lacks, carries its criterion of individuation, what licenses *three dogs*. Grade and subspace are both visible in the object, read off by projection: the Church-style typing that Session 12c found HRR could not have.

The Two Trades

	HRR / MAP (Session 12)	**FGA (wedge)**
binding type	Vec $\to$ Vec $\to$ Vec	Vec $\to$ Vec $\to$ Bivec
the metric	assumed: random atoms, $G \approx I$	declared: a signature
bound pair visible in the type?	no	yes
role versus filler	the codebook; binding commutes	the subspace: $\mathbf{r} \in V_R$, $\mathbf{f} \in V_E$
argument order	via role vectors	in the sign, and via roles
how many things bound?	not visible	the grade
superposition	$+$ in one space	$+$ within a grade
unbinding	approximate inverse, then cleanup	contraction; exact under orthogonality
memory	fixed d	grows with grade: $\binom{d}{k}$
cast to a vector	built in, lossy	explicit `dual`, when wanted

Neither column dominates. HRR's single space is a real advantage for memory and retrieval; the typed representation is a real advantage for composition and for knowing what you have. The position paper states the trade as exact recovery for a bounded number of bindings against approximate recovery for an unbounded number, and argues that argument structure, whose arity is small, suits the exact regime. The cross product is the one place where the two columns coincide, and only because $\binom{3}{2} = 3$.

Possible Final-Project Directions

1. **Capacity of typed vs. untyped binding.** Extend `ga_programs.py` to $n \approx 10$–12 with random role and filler vectors. Bind k role–filler pairs, superpose, unbind, and measure retrieval accuracy as k grows. Do the same with circular convolution at a matched memory budget. Where do the curves cross?
2. **Analogy as rotation.** In a word-embedding space, the usual analogy *king* $-$ *man* $+$ *woman* is a translation. Try it as a rotation instead: build the plane $\mathbf{man} \wedge \mathbf{woman}$ and apply the turn-in-this-plane program to king. Does a rotor do better or worse than a translation, and on which analogy categories?
3. **Order without roles.** Encode two-argument relations (*dog bites man*) with a single wedge and recover the argument order from the sign. Compare with HRR, where the order must be supplied by role vectors. How far does the orientation bit get you before you need roles anyway?

Each of these can be done with the toy library from today plus `numpy`; none needs a GPU.

Part VII
Close

Summary

1. In 3D there is one quarter-turn program per plane, and the plane's i is its unit bivector. Grades are types, readable from the program text.
2. `wedge` is a **constructor**: it compiles two vectors into the program for their plane, with the area as amplitude and the sign as the direction of turn. It is anticommutative, it forgets on purpose, and it is associative.
3. `dual` is a **cast**: it re-encodes a k-blade as its $(n-k)$-dimensional complement. No geometry happens; a global flag, handedness, is involved.
4. The cross product is the **macro** $\texttt{dual}_3 \circ \texttt{wedge}$. Its three oddities are consequences: it type-checks only at $n = 3$; its right-hand rule is the cast's flag; its non-associativity comes from casting mid-pipeline. The triple product behaves because it casts last.
5. The cross product is the one case where a typed binding can be cast back to Vec without loss. HRR gives up the type to keep one space; FGA keeps the grade, binds with the wedge, unbinds exactly by contraction, gives a relation two slots, and casts explicitly. Both are trades.

Coming up in Session 14: the full table of $\mathrm{Cl}(3,0,0)$, rotors at work, contraction as application, and projection.

Exercises

1. By hand, using only SWAP and COLLAPSE, show that $\mathbf{e}_{13}\, I^{-1} = -\,\mathbf{e}_2$ in 3D, with $I = \mathbf{e}_{123}$.
2. Show that $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \mathtt{dual}_3(\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c})$ for arbitrary $\mathbf{a}, \mathbf{b}, \mathbf{c}$, and conclude that the scalar triple product is invariant under cyclic permutation.
3. In the toy library, implement $\mathtt{dual}_4$ and verify that applying it twice to a bivector returns the bivector, not its negative. Explain the sign difference from 3D using $I_4^2 = +1$.
4. Build *dog bites man* and *man bites dog* as single wedges and show they differ by sign. Then build them with circular convolution (`numpy.fft` will do) and show they are identical. What is the minimal addition that lets HRR tell them apart?
5. Write `rotate(v, B, theta)` that turns a vector $\mathbf{v}$ in the plane of a unit bivector B by angle θ, using only gp and a truncated exponential series for the rotor of Session 13b. Check it against a 2×2 rotation matrix when $B = \mathbf{e}_{12}$.

Exercises: Answers

1. $\mathbf{e}_{13}\, I^{-1} = -\,\mathbf{e}_1\mathbf{e}_3\,\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3$. Move the second $\mathbf{e}_1$ left past $\mathbf{e}_3$: one sign. Then $-(-\,\mathbf{e}_1\mathbf{e}_1\,\mathbf{e}_3\mathbf{e}_2\mathbf{e}_3) = \mathbf{e}_3\mathbf{e}_2\mathbf{e}_3 = -\,\mathbf{e}_2\mathbf{e}_3\mathbf{e}_3 = -\,\mathbf{e}_2$. (Equivalently $\mathbf{e}_{13} = -\,\mathbf{e}_{31}$ and $\mathbf{e}_{31}\, I^{-1} = \mathbf{e}_2$.)
2. $\mathbf{b} \times \mathbf{c} = (\mathbf{b} \wedge \mathbf{c})\, I^{-1}$, and for a vector $\mathbf{a}$ and a bivector B the scalar part of $\mathbf{a}\,(B\, I^{-1})$ equals $(\mathbf{a} \wedge B)\, I^{-1}$, because I commutes with everything in 3D and the grade-0 part of $\mathbf{a}\, B\, I^{-1}$ comes from the grade-3 part of $\mathbf{a}B$. So $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \mathtt{dual}_3(\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c})$. The wedge of three vectors is unchanged by a cyclic permutation (two swaps, sign $+$), so the triple product is too.
3. $I_4 = \mathbf{e}_{1234}$ has $I_4^2 = +1$ (six swaps to reverse, sign $+$, then four collapses), so $I_4^{-1} = I_4$ and $\mathtt{dual}_4^2(x) = x\, I_4^2 = x$. In 3D, $I_3^2 = -1$, so two casts return $-x$.
4. $\mathbf{dog} \wedge \mathbf{man} = -\,\mathbf{man} \wedge \mathbf{dog}$ by anticommutativity. Circular convolution commutes, so the two HRR vectors coincide to machine precision. The minimal addition is one role vector: bind $\mathbf{agt} \circledast \mathbf{dog} + \mathbf{pat} \circledast \mathbf{man}$, as Session 12 did.
5. $R = \cos\frac{\theta}{2} - B \sin\frac{\theta}{2}$, computed as the series $\sum_k (-B\theta/2)^k / k!$; return $R\,\mathbf{v}\,\widetilde{R}$. For $B = \mathbf{e}_{12}$ and $\theta = 90°$, $(3, 1) \mapsto (-1, 3)$, matching the matrix $\left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$.

Reading

- Gauss (1831), *Theoria residuorum biquadraticorum, commentatio secunda*: the geometric reading of complex numbers.
- Hamilton (1844), *On quaternions*, Philosophical Magazine. Grassmann (1844), *Die lineale Ausdehnungslehre*.
- Clifford (1878), *Applications of Grassmann's extensive algebra*, American Journal of Mathematics 1.
- Crowe (1967), *A History of Vector Analysis*: Gibbs, Heaviside, and how the cross product won.
- Hestenes (1986), *New Foundations for Classical Mechanics*, Kluwer.
- Dorst, Fontijne and Mann (2007), *Geometric Algebra for Computer Science*, Morgan Kaufmann: chapters 2–3 for the wedge and the dual, and the cross product as $\texttt{dual} \circ \texttt{wedge}$.
- Macdonald (2010), *Linear and Geometric Algebra*: a gentle undergraduate text.
- Plate (1995), *Holographic reduced representations*, IEEE Transactions on Neural Networks 6; Plate (2003), *Holographic Reduced Representation*, CSLI.
- Pustejovsky (1995), *The Generative Lexicon*, MIT Press: type coercion, chapter 7. Pustejovsky (2026), *Toward a Functional Geometric Algebra for natural language semantics*, arXiv:2604.25902v2: slots, wedge binding, exact unbinding.
- Code for today: `ga_programs.py` on the course site.